

Technical Interview Prep — Q&A Guide

Based on your Day-1 tracker: Programming+OOPs, CN, OS, DBMS, DSA, ML, Deep Learning, Agentic AI

1. Programming + OOPs (10)

Q1. What are the 4 pillars of OOP? Encapsulation, Abstraction, Inheritance, Polymorphism.

Q2. Encapsulation vs Abstraction — difference? Encapsulation = hiding data (using private vars + getters/setters). Abstraction = hiding complexity (showing only what's needed via interfaces).

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance # encapsulation (private)

    def deposit(self, amt):
        self.__balance += amt

    def get_balance(self):
        return self.__balance
```

Q3. Types of Inheritance? Single, Multiple, Multilevel, Hierarchical, Hybrid.

Q4. Method Overloading vs Overriding? - Overloading = same method name, different params (compile-time / Python doesn't support natively). - Overriding = child class redefines parent's method (run-time).

```
class Animal:
    def sound(self):
        print("Some sound")

class Dog(Animal):
    def sound(self): # overriding
        print("Bark")
```

Q5. What is a Constructor? Can we overload it? Special method that runs on object creation. In Java/C++ yes, in Python simulate via default args.

Q6. Static vs Instance methods/variables? Static = belongs to class, shared across all objects. Instance = belongs to individual object.

```
class Counter:
    count = 0 # static
    def __init__(self):
        Counter.count += 1
```

Q7. What is Polymorphism? Give example. One interface, many implementations.

```
for shape in [Circle(), Square()]:
    shape.area() # each behaves differently
```

Q8. Abstract class vs Interface? Abstract class = partial implementation allowed. Interface = only method signatures (pure contract). Python uses ABC module for abstract classes.

```
from abc import ABC, abstractmethod
class Shape(ABC):
    @abstractmethod
    def area(self): pass
```

Q9. What is a Deep copy vs Shallow copy? Shallow = copies references (nested objects shared). Deep = fully independent copy.

```
import copy
a = [[1,2],[3,4]]
b = copy.deepcopy(a)
```

Q10. What is Garbage Collection? Automatic memory management — reclaims memory of objects with no references (Python uses reference counting + cyclic GC).

2. Computer Networks (5)

Q1. OSI Model — 7 layers? Physical → Data Link → Network → Transport → Session → Presentation → Application. (“Please Do Not Throw Sausage Pizza Away”)

Q2. TCP vs UDP? | TCP | UDP | |—|—| | Connection-oriented | Connectionless | | Reliable, ordered | Fast, no guarantee | | Used: HTTP, email | Used: video calls, DNS |

Q3. What happens when you type a URL in the browser? DNS lookup → TCP handshake (3-way) → TLS handshake (if HTTPS) → HTTP request → Server response → Browser renders.

Q4. HTTP Status codes? - 200 OK, 201 Created - 301 Redirect - 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found - 500 Internal Server Error

Q5. What is DNS? What is a Subnet? DNS = translates domain name to IP address. Subnet = dividing a network into smaller logical segments for better management/security.

3. Operating Systems (5)

Q1. Process vs Thread? Process = independent program with its own memory space. Thread = lightweight sub-unit of a process, shares memory with sibling threads.

Q2. What is Deadlock? Conditions for it? Two+ processes waiting on each other forever. Conditions: Mutual Exclusion, Hold & Wait, No Preemption, Circular Wait.

Q3. CPU Scheduling Algorithms? FCFS, SJF, Round Robin, Priority Scheduling.

Q4. What is Paging? Memory management scheme — splits memory into fixed-size pages to avoid external fragmentation.

Q5. Semaphore vs Mutex? Mutex = lock, only 1 thread can access at a time (binary). Semaphore = signaling mechanism, can allow N threads (counting).

4. DBMS (10)

Q1. What is Normalization? Explain 1NF, 2NF, 3NF. Process to reduce data redundancy. - 1NF: atomic values, no repeating groups - 2NF: 1NF + no partial dependency on composite key - 3NF: 2NF + no transitive dependency

Q2. ACID Properties? Atomicity, Consistency, Isolation, Durability.

Q3. Types of SQL Joins?

```
SELECT * FROM A INNER JOIN B ON A.id = B.id;
SELECT * FROM A LEFT JOIN B ON A.id = B.id;
SELECT * FROM A RIGHT JOIN B ON A.id = B.id;
SELECT * FROM A FULL OUTER JOIN B ON A.id = B.id;
```

Q4. Primary Key vs Foreign Key vs Unique Key? Primary = uniquely identifies row (no NULL). Foreign = references another table's PK. Unique = no duplicates, but allows one NULL.

Q5. What is an Index? Why use it? Data structure (usually B-Tree) that speeds up SELECT queries at the cost of slower INSERT/UPDATE.

Q6. Clustered vs Non-Clustered Index? Clustered = physically reorders table data (1 per table). Non-clustered = separate structure pointing to data (multiple allowed).

Q7. Write a query: 2nd highest salary.

```
SELECT MAX(salary) FROM Employee
WHERE salary < (SELECT MAX(salary) FROM Employee);
```

Q8. Write a query: Find duplicate rows.

```
SELECT name, COUNT(*)
FROM Employee
GROUP BY name
HAVING COUNT(*) > 1;
```

Q9. What is a Transaction? What is Isolation Level? A transaction is a group of operations executed as a single unit. Isolation levels: Read Uncommitted, Read Committed, Repeatable Read, Serializable — control how much one transaction sees of another's uncommitted changes.

Q10. What are Window Functions? Example (CTE + ROW_NUMBER)

```
WITH ranked AS (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salary
DESC) AS rnk
    FROM Employee
)
SELECT * FROM ranked WHERE rnk = 1; -- top earner per dept
```

5. DSA (20)

Q1. Reverse a Linked List.

```
def reverse(head):
    prev = None
    while head:
        nxt = head.next
        head.next = prev
        prev = head
        head = nxt
    return prev
```

Q2. Detect cycle in a Linked List (Floyd's algorithm).

```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    if slow == fast:
        return True
    return False
```

Q3. Binary Search.

```
def binary_search(arr, target):
    lo, hi = 0, len(arr) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1
```

Q4. Merge Sort (Time: $O(n \log n)$).

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left, right = merge_sort(arr[:mid]), merge_sort(arr[mid:])
    result, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    return result + left[i:] + right[j:]
```

Q5. Find max subarray sum (Kadane's Algorithm).

```
def max_subarray(nums):
    max_sum = cur_sum = nums[0]
    for n in nums[1:]:
        cur_sum = max(n, cur_sum + n)
        max_sum = max(max_sum, cur_sum)
    return max_sum
```

Q6. Check if a string is a palindrome.

```
def is_palindrome(s):
    return s == s[::-1]
```

Q7. BFS and DFS on a Graph.

```
def bfs(graph, start):
    visited, queue, order = {start}, [start], []
    while queue:
        node = queue.pop(0)
        order.append(node)
        for nb in graph[node]:
            if nb not in visited:
                visited.add(nb); queue.append(nb)
    return order

def dfs(graph, node, visited=None):
    if visited is None: visited = set()
    visited.add(node)
    for nb in graph[node]:
        if nb not in visited:
            dfs(graph, nb, visited)
```

```
return visited
```

Q8. Two Sum problem.

```
def two_sum(nums, target):
    seen = {}
    for i, n in enumerate(nums):
        if target - n in seen:
            return [seen[target - n], i]
        seen[n] = i
```

Q9. What is Dynamic Programming? Example — Fibonacci with memoization. DP = breaking problem into overlapping subproblems, storing results to avoid recomputation.

```
def fib(n, memo={}):
    if n <= 1: return n
    if n not in memo:
        memo[n] = fib(n-1, memo) + fib(n-2, memo)
    return memo[n]
```

Q10. Time complexity of common operations (must memorize) |
Structure | Access | Search | Insert | Delete | |---| | Array |
O(1) | O(n) | O(n) | O(n) | | Linked List | O(n) | O(n) | O(1) | O(1) | |
Hash Map | - | O(1) avg | O(1) avg | O(1) avg | | BST (balanced) | O(log
n) | O(log n) | O(log n) | O(log n) |

Q11. Find the missing number in array 1 to N.

```
def missing_number(arr, n):
    return n*(n+1)//2 - sum(arr)
```

Q12. Implement a Stack using Queue (or vice versa).

```
from collections import deque
class StackUsingQueue:
    def __init__(self):
        self.q = deque()
    def push(self, x):
        self.q.append(x)
        for _ in range(len(self.q)-1):
            self.q.append(self.q.popleft())
    def pop(self):
        return self.q.popleft()
```

Q13. Level order traversal of Binary Tree.

```
def level_order(root):
    if not root: return []
    result, queue = [], [root]
    while queue:
        level = []
        for _ in range(len(queue)):
            node = queue.pop(0)
            level.append(node.val)
            if node.left: queue.append(node.left)
            if node.right: queue.append(node.right)
        result.append(level)
    return result
```

Q14. Explain Time & Space complexity of QuickSort. Average: O(n log n), Worst: O(n²) (when pivot is bad, e.g. sorted array). Space: O(log n) due to recursion stack.

Q15. Trapping Rain Water (common FAANG question — know the approach). Two-pointer approach: track max height from left and right, water trapped at each index = min(maxLeft, maxRight) - height[i].

6. Machine Learning (10)

Q1. Supervised vs Unsupervised vs Reinforcement Learning?

Supervised = labeled data (regression/classification). Unsupervised = no labels (clustering). Reinforcement = agent learns via rewards/penalties.

Q2. Bias-Variance Tradeoff? High bias = underfitting (too simple). High variance = overfitting (too complex, doesn't generalize).

Q3. Overfitting — how to prevent it? Regularization (L1/L2), Dropout, Cross-validation, more data, early stopping, simpler model.

Q4. Precision vs Recall vs F1 Score? - Precision = $TP / (TP + FP)$ — how many predicted positives are correct - Recall = $TP / (TP + FN)$ — how many actual positives were found - F1 = harmonic mean of precision & recall

Q5. What is Cross-Validation? Splitting data into k folds, training on k-1 and validating on the remaining fold, rotating — gives a more reliable estimate of model performance.

Q6. Explain Gradient Descent. Optimization algorithm — iteratively updates weights in the direction that reduces the loss function.

```
# w = w - learning_rate * gradient
w = w - lr * dL_dw
```

Q7. Difference: Linear Regression vs Logistic Regression?

Linear = predicts continuous value. Logistic = predicts probability for classification (uses sigmoid function).

Q8. What is Feature Engineering / Scaling? Why needed?

Transforming raw data into useful features. Scaling (StandardScaler/MinMax) needed for algorithms sensitive to magnitude (KNN, gradient descent-based models).

Q9. Random Forest vs Decision Tree? Random Forest = ensemble of many decision trees (bagging), reduces overfitting and variance compared to a single tree.

Q10. Simple ML pipeline code (sklearn):

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2)
model = RandomForestClassifier()
model.fit(X_train, y_train)
preds = model.predict(X_test)
print(accuracy_score(y_test, preds))
```

7. Deep Learning (20)

Q1. What is a Neural Network? Basic structure? Layers of neurons (input → hidden → output) connected by weighted edges, mimics brain-like computation.

Q2. What is Backpropagation? Algorithm to update weights by propagating the error gradient backward from output to input layer using chain rule.

Q3. Activation Functions — Sigmoid, ReLU, Tanh, Softmax?

Sigmoid: (0,1) range, used for binary output, suffers vanishing gradient - ReLU: $\max(0, x)$, most common in hidden layers, avoids

vanishing gradient - Tanh: (-1,1) range, zero-centered - Softmax:
converts logits to probability distribution, used in multi-class output

Q4. Vanishing/Exploding Gradient Problem? In deep networks, gradients shrink (vanish) or grow (explode) through many layers during backprop, making training unstable. Solved via ReLU, batch norm, residual connections, gradient clipping.

Q5. CNN — what is it used for and key components? Used for image data. Components: Convolution layer (feature extraction via filters), Pooling layer (downsampling), Fully Connected layer (classification).

```
import torch.nn as nn
class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 16, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.fc = nn.Linear(16*16*16, 10)

    def forward(self, x):
        x = self.pool(nn.functional.relu(self.conv1(x)))
        x = x.view(x.size(0), -1)
        return self.fc(x)
```

Q6. RNN vs LSTM vs GRU — why LSTM over RNN? RNN struggles with long sequences (vanishing gradient). LSTM/GRU use gates (forget/input/output) to retain long-term dependencies.

Q7. What is Dropout? Regularization technique — randomly disables neurons during training to prevent overfitting.

Q8. Batch Normalization — why used? Normalizes layer inputs to stabilize and speed up training, allows higher learning rates.

Q9. Optimizers — SGD vs Adam? SGD = simple, updates weights using gradient * learning rate. Adam = adaptive learning rate per parameter, combines momentum + RMSProp, converges faster in practice.

Q10. What is Transfer Learning? Using a pretrained model (e.g. ResNet trained on ImageNet) and fine-tuning it on a new, smaller dataset — saves training time and data requirement.

Q11. Loss Functions — MSE vs Cross-Entropy? MSE = used for regression. Cross-Entropy = used for classification, penalizes confident wrong predictions more heavily.

Q12. What is an Epoch, Batch, Iteration? Epoch = one full pass over training data. Batch = subset of data processed at once. Iteration = one weight update (one batch processed).

Q13. Explain Transformers / Self-Attention (high-level). Self-attention lets each token in a sequence weigh the importance of every other token — enables parallel processing of sequences (unlike RNN) and captures long-range dependencies. Basis of BERT, GPT.

Q14. What is Overfitting in DL and how to detect it? Training loss keeps decreasing but validation loss increases/plateaus — model memorizes training data instead of generalizing.

Q15. Simple PyTorch training loop (know this cold):

```
for epoch in range(epochs):
    optimizer.zero_grad()
    output = model(X_train)
    loss = criterion(output, y_train)
    loss.backward()
    optimizer.step()
```

Q16. What is an Embedding layer? Maps discrete inputs (like words/tokens) to dense continuous vectors, learned during training, captures semantic relationships.

Q17. Difference between Encoder and Decoder in Transformers? Encoder processes input sequence into a contextual representation. Decoder generates output sequence (e.g. translation, text generation), often using encoder's output + previously generated tokens.

Q18. What is Learning Rate? What happens if too high/low? Controls step size in gradient descent. Too high → overshoots minimum, diverges. Too low → very slow convergence.

Q19. What is a Confusion Matrix? Table showing TP, FP, TN, FN — used to compute accuracy, precision, recall, F1.

Q20. Gemini 2.5 Flash / LLM basics — what is a Token? Smallest unit of text (word/sub-word) an LLM processes; input/output is measured and billed in tokens, not characters.

8. Agentic AI (20)

Q1. What is an AI Agent? A system powered by an LLM that can reason, plan, and take actions (using tools) to autonomously achieve a goal, rather than just responding to a single prompt.

Q2. What is RAG (Retrieval-Augmented Generation)? Why use it? Combines retrieval (fetching relevant documents via vector search) with generation (LLM produces answer using retrieved context) — reduces hallucination and grounds answers in real data.

Q3. Explain the RAG pipeline (you've built this — know it well).
1. Chunk documents → 2. Generate embeddings → 3. Store in vector DB → 4. On query: embed query, do similarity search → 5. Pass top-k chunks + query to LLM → 6. LLM generates grounded answer.

```
from langchain.text_splitter import RecursiveCharacterTextSplitter
splitter = RecursiveCharacterTextSplitter(chunk_size=500,
chunk_overlap=50)
chunks = splitter.split_text(document_text)
```

Q4. What is a Vector Database? Examples? Stores embeddings and enables fast similarity search (cosine similarity / ANN). Examples: FAISS, Pinecone, Chroma, Weaviate.

Q5. What is LangChain used for? Framework to build LLM apps — chains prompts, manages memory, connects to tools/APIs, and orchestrates agents.

Q6. What is a "Tool" in agentic frameworks? A function the LLM can call (e.g. web search, calculator, API call) to extend its capabilities beyond text generation.

Q7. What is Prompt Engineering? Give an example of a well-structured prompt. Crafting inputs to get better/more reliable LLM outputs — via clear instructions, examples (few-shot), specifying format, and step-by-step reasoning (chain-of-thought).

You are an expert resume reviewer.
Task: Review the resume below and list 3 improvements.
Format: Return as a numbered list.
Resume: {resume_text}

Q8. What is Chain-of-Thought (CoT) prompting? Asking the model to reason step-by-step before giving a final answer — improves accuracy on complex/logical tasks.

Q9. What is the difference between an LLM and an Agent? LLM = generates text based on a prompt (single-shot reasoning). Agent = LLM + memory + tools + planning loop, can take multi-step actions to complete a task.

Q10. What is Function/Tool calling in Gemini/Claude/OpenAI APIs? The model outputs a structured request to call a specific function with arguments; your code executes it and returns the result back to the model to continue reasoning.

Q11. What is context window? Why does it matter for RAG/Agents? Max number of tokens a model can process at once (input + output). Matters because it limits how much retrieved context or conversation history can be passed in a single call.

Q12. What is an embedding? How is similarity measured? A numerical vector representation of text capturing semantic meaning. Similarity typically measured via cosine similarity between vectors.

Q13. Multi-agent systems — what are they? Multiple specialized agents (e.g. researcher agent, writer agent, reviewer agent) collaborating/communicating to solve a larger task, often coordinated by an orchestrator agent.

Q14. What is memory in an agent system (short-term vs long-term)? Short-term = conversation history within a session. Long-term = persisted knowledge/preferences stored across sessions (e.g. in a vector DB or key-value store).

Q15. What is hallucination in LLMs? How does RAG help reduce it? Model generates confident but factually incorrect output. RAG grounds responses in retrieved real documents, reducing reliance on memorized (and possibly wrong) knowledge.

Q16. Explain your Interview Prep Agent / Content Agent Dashboard architecture (be ready to explain your own projects!). Example answer structure: “Built using Streamlit for UI, LangChain for orchestration, Gemini 2.5 Flash as the LLM. [Your agent] takes [input], processes via [steps], and outputs [result]. Used Apify for [scraping/data collection] in the Content Agent project.”

Q17. What is temperature in LLM generation? Controls randomness of output — low temperature (0-0.3) = deterministic/focused, high temperature (0.7-1.0) = more creative/random.

Q18. Zero-shot vs Few-shot vs Fine-tuning? Zero-shot = no examples given. Few-shot = a few examples in the prompt. Fine-tuning = retraining model weights on custom data — more expensive but more tailored.

Q19. What are Guardrails/Safety in agentic systems? Mechanisms to constrain agent behavior — input/output validation, restricted tool permissions, human-in-the-loop approval for risky actions.

Q20. How would you evaluate an LLM/Agent’s output quality? Metrics: relevance, faithfulness/groundedness (for RAG), latency, cost per query, human eval/scoring, and task success rate for agents.

Quick tips before the interview

- Be ready to **explain your own projects** in depth — Interview Prep Agent, Content Agent Dashboard, RAG Chatbot, ATS Resume Builder. Interviewers love drilling into “why did you choose X over Y” for your own work.
- Practice explaining answers **out loud in under 60 seconds** — technical rounds reward clarity over length.

- For DSA, be ready to **code on a whiteboard/shared doc**, not just recite the answer.

Good luck — you've got solid ground covered across all 8 areas. 🍀